

# STAFAN: AN ALTERNATIVE TO FAULT SIMULATION

Sunil K. Jain  
Vishwani D. Agrawal

AT&T Bell Laboratories  
Murray Hill, New Jersey 07974

**ABSTRACT** — STATistical Fault ANalysis (STAFAN) is proposed as an alternative to fault simulation of digital circuits. In this analysis, controllabilities and observabilities of circuit nodes are defined as probabilities which are estimated from signal statistics obtained from fault-free simulation. Special procedures are developed for dealing with these quantities at fanout nodes and at feedback nodes. The computed probabilities are used to derive unbiased estimates of fault detection probabilities and overall fault coverage for the given set of input vectors. Fault coverage and the undetected fault data obtained from STAFAN for actual circuits are shown to agree favorably with the fault simulator results. The computational complexity added to a fault-free simulator by STAFAN grows only linearly with the number of circuit nodes.

## 1.0 INTRODUCTION

Fault simulation is an essential part of VLSI design. Complexity of a fault simulator in terms of the CPU time and the memory requirements is known to grow at least as the square of the number of gates in the circuit [1]. For VLSI circuits, this is a serious limitation. One approach that has been used to combat fault simulation complexity is the random sampling of faults [2]. This requires simulation of a randomly selected subset of faults and gives an estimate of the fault coverage. The accuracy of this estimate depends only upon the number of simulated faults and not on the total number of faults in the circuit. The disadvantages of fault sampling technique are that 1) it assumes the availability (or development) of a complete fault simulation program, and 2) it gives absolutely no information about the detectability of the faults that were not sampled. A list of undetected faults is desirable for test vector development and for checking redundancies.

In this paper we develop a STATistical Fault ANalysis (STAFAN) technique which requires only the fault-free simulation. Fault-free (or true-value) simulation is essential for design verification and is considerably less complex than the fault simulation. The presented analysis calculates the probability of detection for each stuck fault in the circuit from the signal statistics. In a recent work [3] fault detection probabilities were computed for random stimuli. In general, upper and lower bounds for the detection probabilities could be obtained. These bounds could be narrowed down at the cost of greater amount of computation time. However, the method could handle only combinational circuits. In contrast, the present method computes the detection probabilities for the given stimuli (vector set) with which the fault-free simulation is carried out. No elaborate software development is needed. The calculation adds only a small overhead to the fault-free simulation. The method is, in no way, limited to combinational circuits. It produces an estimate of fault coverage by the vector stimuli used in simulation. Since the estimate is statistical, higher accuracy could be expected for larger circuits. Another

technique, known as critical path tracing [4], has been proposed as an alternative to fault simulation but it too can analyze only the circuits that are basically combinational.

STAFAN makes use of the concepts of *controllability* and *observability* [5], [6]. These quantities are redefined as probabilities of controlling and observing the lines. Controllability of a line is estimated by collecting the statistics of activity on that line. Observability is then computed from the estimated controllabilities. The product of the appropriate controllability and observability gives the detection probability of a fault. Faults can be graded in terms of their detection probabilities and the faults with lower detection probabilities are the likely ones to be left undetected by the vector set. A simple calculation leads to fault coverage.

## 2.0 THEORY

Consider the logic model of the circuit as is often used by simulators [7]. In this model the circuit is described in terms of single-output boolean (AND, OR, NAND, NOR and NOT) gates. The gates are connected by paths which are referred to as *lines*. The primary input line values are specified by *input vectors*. Simulator then computes the logical values (0 or 1) for all other lines. We will consider the stuck-type ( $s-a-1$  and  $s-a-0$ ) faults on input and output lines of each gate.

Let us define the following quantities for each line:

- $C1(\ell)$  or *one-controllability* of line  $\ell$  is the probability of line  $\ell$  having a value 1 on a randomly selected vector.
- $C0(\ell)$  or *zero-controllability* of line  $\ell$  is the probability of line  $\ell$  having a value 0 on a randomly selected vector.
- $B1(\ell)$  or *one-observability* of line  $\ell$  is the probability of observing the line  $\ell$  at a primary output when the value of line  $\ell$  is 1. This is the conditional probability of sensitizing a path from line  $\ell$  to a primary output, given that the value of line  $\ell$  is 1. Similarly,
- $B0(\ell)$  or *zero-observability* of line  $\ell$  is the path sensitization probability from line  $\ell$  to a primary output when the value of line  $\ell$  is zero.

### 2.1 Statistical Estimation of Controllabilities

Suppose we simulate  $N$  input vectors and for every line in the circuit keep two counters in the fault-free simulator. These counters will be called the one-counter and the zero-counter, respectively. Whenever a line assumes a value one or zero, its one-counter or the zero-counter is incremented. Most simulators use three values, 0, 1 and unknown ( $X$ ). When the line value is  $X$ , none of the counters is incremented. If the simulator also generates the high-impedance state on a line then the same

counter that was incremented on the previous vector is incremented again. After simulating  $N$  vectors, the one and zero controllabilities are estimated as

$$C1(\ell) = \frac{\text{one-count}}{N},$$

and

$$C0(\ell) = \frac{\text{zero-count}}{N}.$$

## 2.2 Computation of Observabilities

For observability computation the simulator keep an additional sensitization counter for each input line of a gate. This counter is incremented on a vector only if a path from the corresponding line to the gate output is sensitized. Figure 1 gives two examples of input states for which the sensitization counter of the line  $\ell$  will be incremented. After simulating  $N$  vectors the one-level sensitization probability for line  $\ell$  is estimated as

$$S(\ell) = \frac{\text{sensitization-count}}{N}.$$

For observability computation, first the one and zero observabilities  $B1$  and  $B0$  for all primary output lines are set to 1.0. Next the following procedure of propagating observabilities from a gate output to inputs is used until the observabilities of all lines have been obtained.

Consider the four input AND gate shown in Fig. 2. Lines  $i$ ,  $j$ ,  $k$  and  $\ell$  are the input lines and  $m$  is the output line. Since

$$\begin{aligned} C1(m) &= \text{Prob}\{i=1, j=1, k=1, \ell=1\} \\ &= \text{Prob}\{i=1, j=1, k=1 | \ell=1\} \cdot C1(\ell), \end{aligned}$$

therefore  $\text{Prob}\{i=1, j=1, k=1 | \ell=1\} = C1(m) / C1(\ell)$

Since  $B1(\ell)$  is the probability of observing line  $\ell$ , when the value of line  $\ell$  is 1, we have

$$\begin{aligned} B1(\ell) &= B1(m) \cdot \text{Prob}\{i=1, j=1, k=1 | \ell=1\} \\ &= B1(m) \cdot C1(m) / C1(\ell). \end{aligned}$$

Further,

$$\begin{aligned} S(\ell) &= \text{Prob}\{i=1, j=1, k=1\} \\ &= \text{Prob}\{i=1, j=1, k=1, \ell=1\} + \text{Prob}\{i=1, j=1, k=1, \ell=0\} \\ &= C1(m) + \text{Prob}\{i=1, j=1, k=1 | \ell=0\} \cdot C0(\ell) \end{aligned}$$

and therefore

$$\text{Prob}\{i=1, j=1, k=1 | \ell=0\} = \frac{S(\ell) - C1(m)}{C0(\ell)}.$$

Thus,

$$\begin{aligned} B0(\ell) &= B0(m) \cdot \text{Prob}\{i=1, j=1, k=1 | \ell=0\} \\ &= B0(m) \cdot \left[ \frac{S(\ell) - C1(m)}{C0(\ell)} \right]. \end{aligned}$$

Similarly, for OR gate,

$$B1(\ell) = B1(m) \left[ \frac{S(\ell) - C0(m)}{C1(\ell)} \right]$$

$$B0(\ell) = B0(m) \cdot C0(m) / C0(\ell),$$

for NAND gate,

$$B1(\ell) = B0(m) \cdot C0(m) / C1(\ell)$$

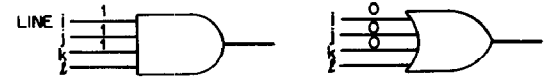


Fig. 1 Sensitization of path from line  $\ell$  to gate output.

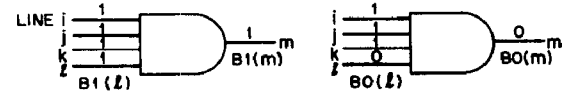


Fig. 2 Propagation of observabilities from output to input of an AND gate.

$$B0(\ell) = B1(m) \left[ \frac{S(\ell) - C1(m)}{C0(\ell)} \right],$$

for NOR gate,

$$B1(\ell) = B0(m) \left[ \frac{S(\ell) - C1(m)}{C1(\ell)} \right]$$

$$B0(\ell) = B1(m) \cdot C1(m) / C0(\ell),$$

and for NOT gate,

$$B1(\ell) = B0(m)$$

and

$$B0(\ell) = B1(m).$$

The above formulas allow propagation of observabilities through logic gates. Two other situations that frequently occur are discussed below.

## 2.3 Fanouts

Consider the line  $\ell$  which fans out into two lines  $i$  and  $j$  as shown in Fig. 3. We will analyze one-observabilities recognizing

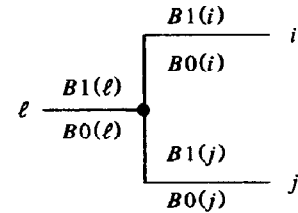


Fig. 3. Observabilities at a fanout.

that similar arguments apply to zero-observabilities.  $B1(i)$  and  $B1(j)$  are the observabilities of lines  $i$  and  $j$ , respectively. The value of line  $\ell$  can be observed through either line  $i$  or line  $j$ . Therefore

$$B1(\ell) \geq B1(i),$$

and

$$B1(\ell) \geq B1(j),$$

or

$$B1(\ell) \geq \max[B1(i), B1(j)].$$

If the observation paths through  $i$  and  $j$  are completely independent then

$$\begin{aligned} B1(\ell) &= B1(i) \cup B1(j) \\ &= B1(i) + B1(j) - B1(i) \cdot B1(j). \end{aligned}$$

This independent path expression represents the upper bound for  $B1(\ell)$ , Thus

$$\max[B1(i), B1(j)] \leq B1(\ell) \leq \bigcup_{k=i,j} B1(k),$$

where  $\cup$  represents the joint probability of the events (assumed independent) whose probabilities are  $B1(i)$  and  $B1(j)$  [8]. Similarly, if line  $\ell$  fans out to  $m$  branches, then,

$$\max_{1 \leq k \leq m} [B1(i_k)] \leq B1(\ell) \leq \bigcup_{k=1}^m B1(i_k).$$

Actual computation of these bounds would require a detailed analysis of the circuit topology. These bounds would depend upon the degree of reconvergence of the fanouts. However, advantage gained from such an analysis would depend upon how wide this range is. For simplicity, we will introduce an arbitrary constant  $\alpha$  such that

$$B1(\ell) = (1-\alpha) \max_{1 \leq k \leq m} [B1(i_k)] + \alpha \left\{ \bigcup_{k=1}^m B1(i_k) \right\}.$$

When  $\alpha = 0$ ,  $B1(\ell)$  is minimum and  $\alpha = 1$  corresponds to the upper bound. In the general case, the value of  $\alpha$  may differ for different fanout branches. Later we will experimentally study the sensitivity of the fault analysis to the value of  $\alpha$  and determine its suitable value for actual circuits.

## 2.4 Feedback

We will demonstrate the calculation of observability in a feedback situation by the NAND flip-flop circuit of Fig. 4. Suppose the observabilities of lines 7 and 8 are known and the controllabilities of lines 1, 3, 4, and 6 are to be determined. Let us assume that the signals on lines 1 and 6 are independent. We will establish an iterative procedure for determining the observabilities of lines 3 and 4. Suppose  $B1_0(4) = 0$  is the zero-order value of one-observability of line 4. Then

$$B1_0(2) = B1(7)$$

$$B0_0(3) = B1(7) \left[ \frac{S(3) - C0(2)}{C0(3)} \right].$$

$B0_0(5)$  is obtained by combining  $B0_0(3)$  and  $B0(8)$  according to the fanout formula. Now

$$B0_0(5) = (1-\alpha) \max \{B0_0(3), B0_0(8)\} + \alpha [B0_0(3) + B0_0(8) - B0_0(3) B0_0(8)]$$

$$B1_1(4) = B0_0(5) \cdot C0(5)/C1(4).$$

$B1_1(2)$  will be obtained by combining  $B1_1(4)$  and  $B1(7)$  in the fanout formula and we proceed further in the above manner to calculate  $B0_1(3)$ ,  $B0_1(5)$ ,  $B1_2(4)$ ,  $\dots$  until the successive iterations produce identical values. It can be verified that such iterations will converge to a unique value. Thus  $B1(4)$  and  $B0(3)$  are computed. Similarly, starting with  $B0_0(4) = 0$ , the values of  $B0(4)$  and  $B1(3)$  can be calculated. Once these are available, observabilities of lines 1 and 6 follow easily.

The above iterative procedure can be applied to feedback structures containing any number of gates. The iterations, in fact, correspond to expanding the structure of Fig. 4 by cutting the feedback loop at line 4 and tracing the circuits along the lines 2, 3 and 5 until we again arrive at 4 at which point the structure consisting of lines 4, 2, 3, 5, 4, is duplicated as shown in Fig. 5. In some simple cases the observability of the signals on the feedback path can be analytically computed. In such cases the iterative procedure is not necessary. Suppose after  $n$ -th iteration  $B1_n(4) = x$ . If  $n$  is sufficiently large such that

$$B1_{n+1}(4) = B1_n(4) = x,$$

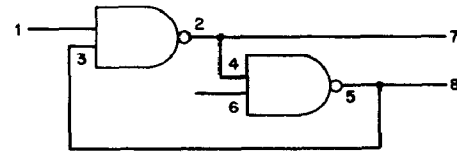


Fig. 4 NAND flip-flop.

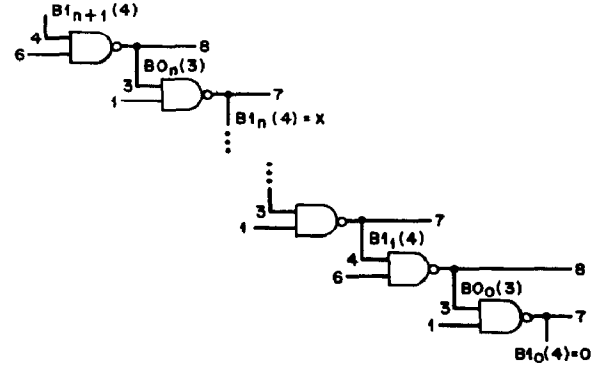


Fig. 5 Iterative analysis

then we can write an expression for  $B1_{n+1}(4)$  in terms of  $x$  and solve the above equation for  $x$ .

In the above analysis of feedback, we assumed that the signals 1 and 6, which sensitize the feedback loop, are independent. Sensitization of the loop corresponds to storing the values of signals in the feedback (provided the number of inversions around the loop is even; for odd number of inversions the loop will give rise to oscillations and the fault analysis will not be valid). In a clocked circuit, the signals 1 and 6, which are derived from the same clock signal, can not be assumed to be independent. In order to adequately analyze such a circuit, we must calculate the *loop sensitization probability*,  $L$ . This can be done by identifying all the input signals of the gates forming a loop and removing from them the output signals of these gates which form the feedback. For example, in Fig. 4, the input signals are 1, 3, 4 and 6. If we remove the output signals, 3 and 4, we are left with 1 and 6. In order to sensitize the loop, 1 and 6 should be simultaneously set to "one". In the fault-free simulator if we set up a loop-counter for each loop to count the occurrences of loop sensitization states, then the loop sensitization probability can be estimated after simulation of  $N$  vectors as

$$L = \frac{\text{loop-count}}{N}.$$

Now  $L$  is the probability with which all inverting gates in the loop functionally become inverters and all non-inverting gates become simple buffers. This is shown for the NAND flip-flop in Fig. 6. The iterative structure in Fig. 6(b) shows that line 4, after  $n$  vectors, can be observed either through line 7 or through line 8 with one inversion or through the previous  $(n-1)$  vectors. This observability, which is shown in Fig. 6(c), will be extended to next vector,  $(n+1)$ th, if the path through the cross-hatched AND gate is sensitized. Node  $4^*$  represents the observability after  $(n+1)$  vectors, if the loop is sensitized. For this hatched gate the probability of sensitization is  $L$ . Again the observability  $x$  of line 4 can be obtained by solving

$$B1_{n+1}(4) = L B1_{n+1}(4^*) = x.$$

where  $B1_{n+1}(4^*)$  is the observability due to three fan out observabilities,  $B1(7)$ ,  $B0(8)$  and  $x$ .

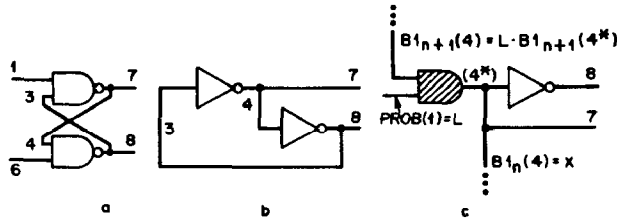


Fig. 6 Model for NAND flip-flop with sensitized loop.

The value of  $x$  can be analytically computed as follows:

$$L \cdot B1_{n+1}(4^*) = x$$

$$L \cdot [K + x - Kx] = x$$

where,

$$K = (1-\alpha)\max[B1(7), B0(8)]$$

$$+ \alpha [B1(7) + B0(8) - B1(7)B0(8)],$$

therefore

$$x = \frac{L \cdot K}{L \cdot K + (1 - L)}.$$

## 2.5 Computation of Fault Detection Probability

Consider the fault "line  $\ell$  stuck-at-1". To detect this fault the line  $\ell$  must be set to a value 0 and then its value should be observed at a primary output. Thus the probability of detection of this fault is given by

$$D1(\ell) = \text{Prob}\{\text{line } \ell \text{ set to 0 and observed}\}$$

$$= \text{Prob}\{\ell \text{ is set to 0}\} \cdot \text{Prob}\{\text{line } \ell \text{ observed} \mid \ell \text{ is set to 0}\}$$

$$= B0(\ell) \cdot C0(\ell).$$

Similarly, the probability of detecting the fault "line  $\ell$  stuck-at-0" can be calculated as

$$D0(\ell) = B1(\ell) \cdot C1(\ell).$$

If for a given fault the detection probability is  $x$ , then the probability of detecting this fault by a set of  $N$  vectors is  $X(N) = 1 - (1-x)^N$ .

## 2.6 Unbiasing Fault Detection Probability

Suppose we simulate  $N$  vectors and from the statistical data collected during simulation obtain the detection probabilities of all the faults. The probabilities thus computed are statistical estimates and can be regarded as random variables whose values converge to their exact values as  $N$  tends to infinity. Let us assume that  $p(x)$  is the probability density function of the detection probability of certain fault. Also assume that the mean value,  $x_0$ , of this random variable is the exact value of the detection probability. Since we do not know the exact probability distribution function, for simplicity, we assume that  $x$  is uniformly distributed in the domain  $[x_0 - \Delta, x_0 + \Delta]$ . The estimated probability of detecting the fault by at least one among  $N$  vectors is given by the random variable,

$$1 - (1-x)^N$$

In order to find the actual probability,  $1 - (1-x_0)^N$ , we proceed by computing the expected value,

$$E(1 - (1-x)^N) = 1 - E(1-x)^N = 1 - \frac{1}{2\Delta} \int_{x_0-\Delta}^{x_0+\Delta} (1-x)^N dx$$

$$E(1 - (1-x)^N) = 1 - \frac{1}{2\Delta(N+1)} [(1-x_0+\Delta)^{N+1} - (1-x_0-\Delta)^{N+1}]$$

$$= 1 - (1-x_0)^N \left[ 1 + \frac{N(N-1)}{3!} (1-x_0)^{-2} \Delta^2 + \dots \right]$$

The above expression shows that the mean value of our estimation is biased away from the required value  $1 - (1-x_0)^N$ . In order to remove this bias, we modify the estimation as follows:

$$X(N) = 1 - \frac{(1-x)^N}{W(x_0)}$$

where

$$W(x_0) = 1 + \frac{N(N-1)}{3!} (1-x_0)^{-2} \Delta^2 + \dots$$

If the detection probability was determined through  $N$  Bernoulli trials ([8], pages 495-496) then its standard deviation would be  $\sqrt{x_0(1-x_0)}/\sqrt{N}$ . Let us assume that

$$\Delta = \beta \sqrt{\frac{x_0(1-x_0)}{N}}$$

where  $\beta$  is a constant of proportionality which should account for the determining process not being Bernoulli trials and for the assumed uniform density function for  $x$ . Thus

$$W(x_0) = 1 + \frac{N-1}{6} \beta^2 \frac{x_0}{1-x_0} + \dots$$

Finally, since the true mean  $x_0$  is not known in practice we substitute the sample mean  $x$  in its place to obtain

$$X(N) \approx 1 - \frac{(1-x)^N}{W(x)},$$

leaving the value of  $\beta$  to be determined empirically.

## 2.7 Fault Coverage

Consider a circuit with  $K$  faults and  $V$  test vectors. We divide the set of vectors in  $k$  subsets, each containing  $V/k = N$  vectors. We will calculate the fault coverages  $F(N), F(2N), \dots, F(kN)$ . For the  $i$ -th fault, we compute the detection probabilities from the  $N$ -vector statistics in each subset as  $x_{i1}, x_{i2}, \dots, x_{ij}, \dots, x_{ik}$ . The cumulative detection probability of this fault is given by

$$f_i(jN) = f_i\{(j-1)N\} + [1-f_i\{(j-1)N\}] \left[ 1 - \frac{(1-x_{ij})^N}{W(x_{ij})} \right]$$

$$= 1 - [1-f_i\{(j-1)N\}] \frac{(1-x_{ij})^N}{W(x_{ij})}.$$

Since  $f_i(0) = 0$ , iteratively writing  $f_i(N), f_i(2N), \dots$ , we get

$$f_i(jN) = 1 - \prod_{m=1}^j \frac{(1-x_{im})^N}{W(x_{im})}.$$

Now the fault coverage for the whole circuit is obtained as

$$F(jN) = \frac{1}{K} \sum_{i=1}^K f_i(jN).$$

## 2.8 Complexity of STAFAN

STAFAN requires two types of operations to be included in a fault-free simulator. The first type, involving the updating of counters, must be performed on every vector. The other type, which include computation of controllabilities, observabilities, etc., are performed once every  $N$  vectors, where  $N$  can be

chosen arbitrarily. In general,  $N$  can be large. In the extreme case, the second type of operations may be performed only once after simulating through all the vectors. Also the operations like loop identification and the initialization of counters are performed only once during preprocessing. Thus the main overhead due to STAFAN in the fault-free simulator is due to the first type of operations. These operations are listed in Table 1. Each counter is updated after examining certain line values. The number,  $G$  is the total number of gates in the circuit. Every gate output line has a 0-counter and a 1-counter. Each counter requires two operations: 1) examine line value and 2) update counter. Thus there are  $2G$  operations for each type of counter. In practice, some of these operations may overlap. If  $n_f$  is the average number of inputs per gate then the number of sensitization counters is  $n_f G$ . In order to update this counter for a gate-input line, all other inputs of this gate must be examined. The total numbers of operations connected with these counters is  $n_f^2 G$ . Updating of a loop counter requires examining whether or not certain sensitization counter of each of the gates in the loop was updated. In general, the number of loops in a circuit will be less than  $G$ . If  $n_\ell$  is the average number of gates in a loop, the number of operations associated with these counters will be less than  $(n_\ell + 1)G$ . The total overhead in the fault-free simulator due to all the four types of counters is

$$\text{overhead} \propto G(4 + n_f^2 + n_\ell + a)$$

where  $a < 1$ .

**Table 1. Operations performed in STAFAN**

| Counter               | Required for          | Number of values examined | Number of counters | Number of operations |
|-----------------------|-----------------------|---------------------------|--------------------|----------------------|
| 0-counter             | every gate            | 1                         | $G$                | $2G$                 |
| 1-counter             | every gate            | 1                         | $G$                | $2G$                 |
| sensitization-counter | every gate-input line | $n_f - 1$                 | $n_f G$            | $n_f^2 G$            |
| loop-counter          | every loop            | $n_\ell$                  | $< G$              | $< (n_\ell + 1)G$    |

### 3.0 IMPLEMENTATION

In an experimental study STAFAN algorithms were implemented in an existing MOS simulator [9]. Since this

simulator is also capable of simulating circuits at gate level with  $s-a-0$  and  $s-a-1$  faults, it was possible to compare its fault simulation results with STAFAN fault coverage estimation.

In the implementation it was necessary to set the values of the empirical constants  $\alpha$  and  $\beta$ . For calculation of observabilities at fanouts, suitable value of constant  $\alpha$  was determined by an experiment conducted on a large circuit. A value  $\alpha = 1$  showed good agreement with the fault simulation result. Notice that this corresponds to the assumption that the fanout branches are independent. In fact the results were rather insensitive to the actual value of  $\alpha$ . The other empirical constant  $\beta$  is used for unbiasing the fault detection probabilities. The following approximation was used:

$$X(N) \approx 1 - \frac{(1-x)^N}{W(x)}$$

where

$$W(x) \approx 1 + \frac{N-1}{6} \beta^2 \frac{x}{1-x}$$

Again by experimenting with an actual circuit it was found that  $\beta^2/6 = 5.0$  produced a good match with fault simulation result. The values of  $\alpha$  and  $\beta$  which were thus set in the analysis, upon trials on several other circuits, produced consistently good results. In this implementation the feedback in the cross-coupled gates was handled using the loop-sensitization probability. But the larger feedback loops were processed by the iterative procedure.

### 4.0 RESULTS

Results obtained by STAFAN on several circuits varying in size from 102 gates to 2,723 gates are summarized in Table 2. The plots of fault coverage versus number of vectors for two of the circuits are shown in Figs. 7 and 8. Figure 7 shows the fault coverage plot for a 64-bit ALU (Arithmetic-Logic Unit). Figure 8 shows the fault coverage for the sequential circuit with 1983 gates. From the graphs it can be seen that the STAFAN fault coverage estimation closely follows the actual fault coverage obtained from fault simulator. Different periods ( $N$ ) of analysis were taken for various circuits. It was found that this period of analysis has very small effects on the final fault coverage estimates. From the data in the Table 2, it is evident that STAFAN is a very effective technique for small to large circuits. Average deviation of STAFAN fault coverage from the actual fault simulator result varies from 0.29% to 3.65%.

**TABLE 2 - STAFAN RESULTS**

| CIRCUIT          | #GATES | PRIM. INPUTS/OUTPUTS | FLIP-FLOPS | Number of Vectors | Number of Faults | Period of estimation in STAFAN (N) | FINAL COVERAGE |          |                    |
|------------------|--------|----------------------|------------|-------------------|------------------|------------------------------------|----------------|----------|--------------------|
|                  |        |                      |            |                   |                  |                                    | FAULT SIM. %   | STAFAN % | Average Difference |
| 4-bit ALU        | 102    | 14/8                 | 0          | 52                | 263              | 10                                 | 96.57          | 96.25    | 0.47               |
| 64-bit ALU       | 1827   | 134/69               | 0          | 155               | 4376             | 20                                 | 75.3           | 75.09    | 0.29               |
| 4-bit MULTIPLIER | 359    | 12/10                | 15         | 1111              | 741              | 100                                | 93.12          | 86.80    | 3.37               |
| CIRCUIT A        | 1983   | 38/34                | 86         | 3842              | 5060             | 200                                | 86.44          | 83.61    | 1.24               |
| CIRCUIT B        | 2723   | 64/64                | 98         | 3636              | 5856             | 200                                | 71.26          | 72.57    | 3.65               |

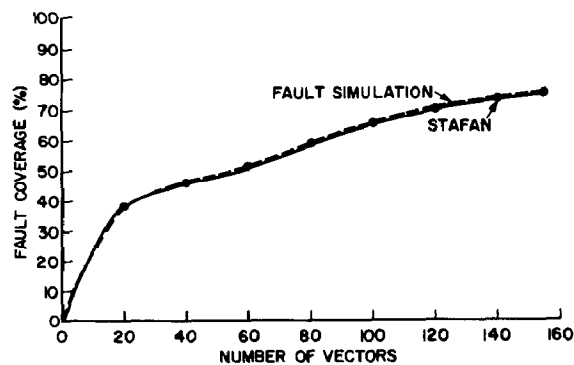


Fig. 7 Fault coverage for a 64-bit ALU.

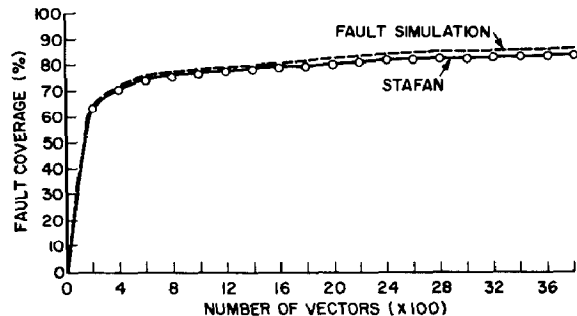


Fig. 8 Fault coverage for CIRCUIT A, a sequential circuit with 1983 gates.

In order to determine how accurately the detection probabilities would predict the undetected faults, all the 4,376 faults of the 64-bit ALU were graded in terms of their cumulative detection probabilities as obtained after simulating through 155 vectors. The faults ordered in terms of decreasing detection probability are shown in Fig. 9. Since the estimated fault coverage was 75.09 percent, the first 3,286 faults in this graded list were assumed to be detected. This left the 1,090 faults, with detection probabilities less than 0.53, as undetected. Comparison with fault simulation results showed that of these 1,090 faults, 1,036 were indeed undetected. Also among the 3,286 faults that were assumed to be detected by STAFAN, only 46 faults were actually reported as undetected by fault simulator.

## 5.0 CONCLUSION

We started this paper with a promise to present an alternative to fault simulation which can handle large circuit complexity. We believe we have kept that promise. Preliminary trials have indicated good results not only for fault coverage but also for finding the faults that remain undetected by a vector set. There are, however, two parameters,  $\alpha$  and  $\beta$ , used in the analysis which must be determined empirically. While  $\beta$  can be determined by a comparison with fault simulation results,  $\alpha$  may, in general, depend upon the structure of circuit. Fortunately, in the experiments that we have conducted, the results seem to be only mildly affected by the value of  $\alpha$ .

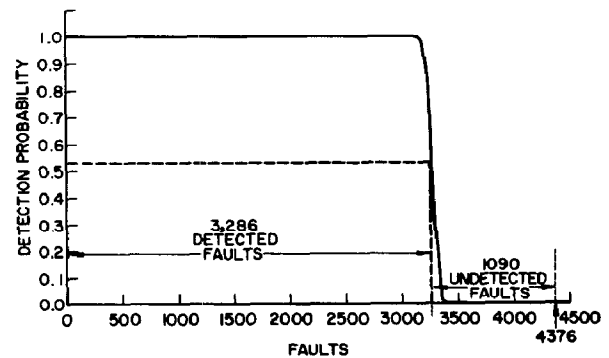


Fig. 9 Detection probabilities for faults in 64 bit ALU after simulating 155-vectors.

## REFERENCES

- [1] P. Goel, "Test generation costs analysis and projections," *Proceedings of 17th Design Automation Conference*, Minneapolis, Minnesota, June 23-25, 1980, pp. 77-84.
- [2] V. D. Agrawal, "Sampling techniques for determining fault coverage in LSI circuits," *Journal of Digital Systems*, Vol. V, pp. 189-202, Fall 1981.
- [3] J. Savir, G. Ditlow, and P. H. Bardell, "Random Pattern Testability," Thirteenth Annual Fault-Tolerant Computing Symposium, Milano, Italy, June 28-30, 1983, pp. 80-89.
- [4] M. Abramovici, P. R. Menon, and D. T. Miller, "Critical Path Tracing — An Alternative to Fault Simulation," *Proceedings of 20th Design Automation Conference*, Miami Beach, Florida, June 27-29, 1983, pp. 214-220.
- [5] J. E. Stephenson and J. Grason, "A testability measure for register transfer level digital circuits," Sixth International Fault Tolerant Computing Symposium, Pittsburgh, Pennsylvania, June 1976, *Digest of Papers*, pp. 101, 107.
- [6] L. H. Goldstein, "Controllability/Observability Analysis of Digital Circuits," *IEEE Transactions on Circuits and Systems*, Vol. CAS-26, September 1979, pp. 685-693.
- [7] S. G. Chappell, C. H. Elmendorf, and L. D. Schmidt, "LAMP: Logic-Circuit Simulator," *Bell System Technical Journal*, Vol. 53, October 1974, pp. 1451-1476.
- [8] K. S. Trivedi, *Probability and Statistics with Reliability, Queueing, and Computer Science Applications*, Englewood Cliffs, N.J.: Prentice-Hall, 1982.
- [9] A. K. Bose, et al., "A Fault Simulator for MOS LSI Circuits," *Proceedings of 19th Design Automation Conference*, Las Vegas, Nevada, June 14-16, 1982, pp. 400-409.